Project 5.1 — TripCraft-Pro-Production-Itinerary-Service

A practice twin of Project 5 — ContentForge Pro. Same skills — take a working multi-agent demo and make it something you'd actually run for real users: a **FastAPI** service in front of it, a **database** that persists every run, a **Docker** image, and an **evaluation suite** that scores quality and catches a regression before it ships. It builds directly on **Project 4.1 — TripCraft**, exactly as Project 5 builds on Project 4. **There are no steps here on purpose.** Read the story, build it yourself, and only open the hints or the solution if you get stuck.

Type: Production multi-agent service (the TripCraft team behind an API) · **Stack:** Python, FastAPI, SQLite, Docker, an LLM-judge eval suite, LangGraph/LangChain · **Level:** Upper-Intermediate

> **Builds on Project 4.1 (TripCraft)** — you start from your finished TripCraft team and add the production layer around it. **Stuck, or want to check your work?** Full hints · Complete solution

The Story — what you are going to build

You built TripCraft (4.1): a supervisor-led team that turns a destination into a reviewed, sourced itinerary. It works — in a terminal, on your machine, when you run it by hand. That is a **demo**. The moment someone else wants to use it — a travel site calling it from their backend, a teammate hitting it from a script, a product manager asking "what did it produce last Tuesday for Tokyo?" — the demo isn't enough. You need it to be a **service**.

You are going to build **TripCraft Pro**: the same itinerary team, made production-ready. Four things turn the demo into something real:

- A **web API** (FastAPI). Instead of typing into a terminal, a caller `POST`s a destination to `/itineraries` and gets back a run id and the itinerary as JSON. A `/health` endpoint lets a load balancer check it's alive, and `GET /itineraries/{id}` fetches any past run.
- A **database** (SQLite). Every run is saved — destination, status, the itinerary, when it ran — so "what did it produce for Tokyo last Tuesday?" is a query, not a shrug. The API reads and writes it; the terminal app still works too.
- A **Docker image**. The whole thing — code, dependencies, the way it starts — packaged so it runs the same on your laptop, a teammate's, and a server. "Works on my machine" stops being a risk.
- An **evaluation suite** (an LLM judge). A fixed set of destinations, each with checks, scored automatically, with a **pass rate** and a **regression check** that deliberately breaks a prompt and

confirms the score drops — so a change that quietly makes itineraries worse can't ship unnoticed.

The one lesson of this project: the gap between "it works when I run it" and "others can rely on it" is a *door* (an API), a *memory* (a database), a *box* (a container), and a *scorecard* (evals). Productionizing is adding those four, without touching the agents inside.

How it works, from a request to a stored, scored itinerary (picture the whole machine)

Read this once and the whole project should form in your head.

The TripCraft **team is unchanged** — it's still the supervisor + research + analysis + writer + reviewer graph from 4.1. What's new is everything *around* it. There's one **pipeline** function — the single door into the team — that takes a destination, runs the graph, and returns the itinerary and status. Both the terminal app and the API call that one function, so the team has exactly one entry point.

A caller sends `POST /itineraries {"destination": "Paris"}`. The route validates the request with a Pydantic model, calls the pipeline, **saves the run to SQLite** (destination, status, itinerary, timestamp) with a new id, and returns the id plus the result as JSON. Later, `GET /itineraries/{id}` reads that row straight back. `/health` returns `{"status": "ok"}` for a monitor. If the destination is unknown, the pipeline returns the honest "nothing known" status, the API still stores the run and returns `200` with that status — a known outcome, not an error.

The whole service is wrapped in a **Dockerfile**: install the requirements, copy the code, and start `uvicorn` on the API. `docker build` + `docker run` and it's up, identically, anywhere.

Separately, the **eval suite** runs the pipeline over a fixed list of destinations, and an **LLM judge** scores each itinerary against written checks (realistic, sourced, covers the main attractions). It reports a **pass rate**. The **regression check** runs the suite once normally, then again with a deliberately sabotaged writer prompt, and confirms the pass rate **drops** — proving the suite actually detects worse output, so it can gate a release.

That's the whole machine: **one pipeline door into the unchanged team → an API that validates, runs, stores, and returns JSON → a database that remembers every run → a Docker box that runs it anywhere → an eval suite + judge that scores quality and catches a regression.**

What you are building (the pieces)

- A **pipeline** (`run_pipeline(destination)`): the one function both the API and the terminal call.
- A **FastAPI app**: `/health` , `POST /itineraries` , `GET /itineraries/{id}` , with Pydantic request/response models.
- A **SQLite layer**: save a run, fetch a run by id.

- A **Dockerfile** that builds the service image and starts uvicorn.
- An **eval suite**: fixed tasks + checks, an LLM judge, a pass rate, and a regression check that breaks a prompt and confirms the score falls.
- The **TripCraft team**, carried in unchanged from 4.1.

Inputs: a destination (over HTTP or the terminal). **Outputs:** a stored, fetchable itinerary as JSON; an eval report with a pass rate. **Rules:** the agents don't change; every run is persisted; the eval suite can actually catch a regression.

A real example to build and test against

- `POST /itineraries {"destination": "Paris"}` → `200`, a run id, status `done`, and the itinerary with its `[source]` tags.
- `GET /itineraries/{that id}` → the same run read back from SQLite.
- `POST /itineraries {"destination": "Atlantis"}` → `200`, status `unknown`, stored all the same.
- `run_eval` over the fixed destinations → a pass rate; the regression check → the sabotaged prompt's pass rate is clearly lower.
- `GET /health` → `{"status": "ok"}`.

How to practice this

1. Read this story. **Don't open the solution yet.** Make sure your TripCraft (4.1) runs first — this project wraps it.
2. Plan it on paper: the one pipeline door, the three routes, the run table, and how the eval judge scores.
3. Build it yourself. A good order: the pipeline + a `/run` route → the SQLite layer + persist every run → the Dockerfile → the eval suite + judge + regression check. Your call.
4. Test it on the real example: a POST that stores and returns, a GET that reads it back, an unknown destination, and the regression check catching a broken prompt.
5. **Only if stuck**, read the hints. **To compare**, read the complete solution.

Why this project is useful (read this at the end)

This is the step every real AI feature has to make: from "it works in a notebook" to "a team depends on it." The four pieces — an API door, a database memory, a container box, an eval scorecard — are exactly what separates a demo from a product, and they're the same four whether the thing inside is a travel-itinerary team, a support bot, or a document pipeline. The habit that matters most is the eval suite with a regression check: it's how you change a prompt or a model next month **without** quietly

shipping worse output, because the scorecard fails the release first. Build TripCraft Pro and you've productionized a multi-agent system end to end — the thing most portfolios never show.

Done when

- `POST /itineraries` runs the team, stores the run, and returns the itinerary as JSON; `GET /itineraries/{id}` reads it back; `/health` is green.
- An unknown destination is stored and returned as a known `unknown` status, not an error.
- `docker build` + `docker run` brings the service up and `/health` responds.
- The eval suite reports a pass rate, and the regression check shows a deliberately broken prompt scoring lower.
- Your tests pass: the API routes (team faked, no key), the DB round-trip, the carried team's unit tests, and the live judge.

The full build, every file, is in the complete solution — use it to check your work, not to skip the thinking.